USER MANUAL

Hardware-Aware QAT Training & Verification

LeNet-5 INT8 Quantisation-Aware Training for FPGA Deployment

Companion to: *FPGA-Accelerated LeNet MNIST Inference — Python Firmware User Manual*

Framework: PyTorch (training) + NumPy (verification)
Hardware Target: PYNQ-ZU FPGA Accelerator (INT8)
Dataset: MNIST Handwritten Digit Recognition

Table of Contents

1. Introduction & Purpose 3
2. Why Conventional Training Does Not Work Here 4
3. The Hardware Contract — Fixed Constants 5
4. Training Setup & Dependencies 7
5. Code Walkthrough — Training 9
   5.1 INT8 Straight-Through Estimator (STE) 9
   5.2 FPGA Requantisation Function 11
   5.3 QuantConv2d and QuantLinear Layers 13
   5.4 LeNet_INT8_QAT Model Architecture 15
   5.5 Dataset Preparation 17
   5.6 Training Loop 18
   5.7 Exporting Weights to .npz 20
6. Code Walkthrough — Software Inference 22
   6.1 Loading Weights 22
   6.2 INT8 Convolution & Helpers 23
   6.3 Full Inference Pipeline 25
   6.4 Per-Digit Accuracy Reporting 27
7. API Reference 28
8. Running the Full Flow 31
9. Troubleshooting 33

1. Introduction & Purpose

This manual documents the training pipeline used to produce the INT8 quantised LeNet-5 model that runs on the FPGA hardware accelerator described in the companion firmware manual. It also documents the software-only inference script used to verify that the exported model is correct before deploying it to the FPGA board.

If you are reading this, you may want to:

- Retrain the model from scratch (for example, to improve accuracy or adapt to a different dataset).
- Understand why the training code looks different from a standard PyTorch tutorial.
- Verify that a newly exported weights file produces correct predictions before programming the FPGA.
- Understand how the software inference script mirrors what the FPGA hardware actually does.

The Two Scripts Covered in This Manual

Script	Purpose
`lenet_qat_train.py`	Trains the LeNet model using hardware-aware Quantisation-Aware Training (QAT) and exports the INT8 weights to a .npz file.
`lenet_numpy_inference.py`	Runs the exported .npz weights through a pure NumPy inference pipeline that exactly mirrors the FPGA hardware behaviour, to verify accuracy before FPGA deployment.

> **NOTE:** *This manual is a companion to the FPGA-Accelerated LeNet MNIST Inference — Python Firmware User Manual. You do not need to read that manual first, but it provides helpful context on how the exported weights are used on the FPGA board.*

2. Why Conventional Training Does Not Work Here

A standard PyTorch LeNet trained in the usual way (floating-point weights, standard normalisation) will NOT work correctly when deployed to this FPGA accelerator. This section explains why.

Standard Training vs. Hardware-Aware Training

Standard PyTorch Training	This Hardware-Aware QAT Training
`Weights stored as float32`	Weights quantised to INT8 during training
`Activations are float32`	Activations quantised to INT8 at every layer
`Input scaled to [0.0, 1.0]`	Input scaled to [0.0, 127.0] to match FPGA input
`Standard MNIST normalisation (mean 0.1307, std 0.3081)`	No standard normalisation — raw pixel scale used
`Accumulations stay in float32`	Accumulations rescaled using fixed integer multiply + shift
`No hardware constants in training`	SCALE_FACTOR, ROUND_FACTOR, SHIFT are locked to match FPGA logic

The Core Problem: Quantisation Error

The FPGA hardware accelerator works exclusively with 8-bit integers (INT8). Every multiplication, accumulation, and activation in the hardware is performed in integer arithmetic. If a model is trained in floating point and then simply rounded to INT8 at export time, the rounding errors compound across layers and accuracy drops significantly.

> **KEY CONCEPT:** *Quantisation-Aware Training (QAT) solves this by simulating the integer rounding during training itself, so the model learns weights that are robust to quantisation error before they are ever deployed to hardware.*

The Hardware Contract

This FPGA accelerator imposes specific constraints that the training code must match exactly. These are not adjustable — they are fixed by the hardware design:

- All weights and activations must be in the range [-128, 127] (signed INT8).
- After each convolution or fully-connected layer, the INT32 accumulation must be rescaled back to INT8 using a specific multiply-then-shift formula.

- The rescaling formula uses three fixed constants: SCALE_FACTOR = 258, ROUND_FACTOR = 16384, SHIFT = 15.
- Input images must be scaled to the range [0, 127], not [0, 1] or normalised with MNIST mean/std.

IMPORTANT: *If you retrain the model and change any of the three constants (SCALE_FACTOR, ROUND_FACTOR, SHIFT), the FPGA hardware will produce wrong results because its internal rescaling logic uses the values shown above. These constants are locked by the hardware design.*

3. The Hardware Contract — Fixed Constants

This section explains the three hardware constants in detail. Understanding them is the most important thing to grasp before reading the training code.

The Three Constants

```
SCALE_FACTOR = 258
ROUND_FACTOR = 16384
SHIFT        = 15
QMIN         = -128
QMAX         =  127
```

Constant	Explanation
`SCALE_FACTOR = 258`	A multiplier applied to the INT32 accumulation. It scales the result into the right magnitude range before the bit-shift. 258 is approximately 2^8 / 1.0, chosen so that after the shift the output fits in INT8.
`ROUND_FACTOR = 16384`	An offset added before the bit-shift. This is 2^(SHIFT-1) = 2^14 = 16384, which provides correct rounding (equivalent to adding 0.5 before truncation). Without this, the shift always rounds down.
`SHIFT = 15`	The number of bits to right-shift after scaling. Dividing by 2^15 = 32768 undoes the scaling and brings the result back into INT8 range. Combined with SCALE_FACTOR=258, the effective divisor is 32768/258 ≈ 127.
`QMIN = -128, QMAX = 127`	The minimum and maximum values for signed 8-bit integers. All values are clipped to this range after rescaling.

The Rescaling Formula — Step by Step

After every convolution or fully-connected operation, the INT32 accumulation is rescaled back to INT8 using this exact sequence of operations:

```
# Step 1: Multiply by SCALE_FACTOR
acc = acc * 258

# Step 2: Add ROUND_FACTOR (for correct rounding)
acc = acc + 16384

# Step 3: Right-shift by SHIFT bits (integer divide by 32768)
acc = acc >> 15

# Step 4: Clip to INT8 range
acc = clip(acc, -128, 127)
```

Why This Formula Is Used

The FPGA hardware cannot perform floating-point division. To divide by a constant, hardware designers use a technique called fixed-point multiplication: instead of dividing by X, multiply by a scaled reciprocal and then shift right. This is faster and uses less hardware logic than a full divider.

The formula above effectively computes:

```
output = round(acc * (258 / 32768))
       = round(acc * 0.007874...)
       ≈ round(acc / 127)
```

This makes sense: if inputs and weights are both in the INT8 range (roughly -127 to 127), then a single multiply produces values up to 127 * 127 = 16129, and summing over a 5x5 = 25 element kernel gives up to about 403,000. Dividing by 127 (per input) per channel brings this back to the INT8 range.

Training vs. Hardware: One Key Difference

In the training code, the bit-shift is replaced by a floating-point division:

```
# During training (floating-point, for gradient flow):
acc = acc / (2 ** SHIFT)   # Divides by 32768.0 — gradients flow through

# During hardware execution (integer, no gradients needed):
acc = acc >> SHIFT          # Integer right-shift by 15 bits
```

> **KEY CONCEPT:** *Using division instead of integer bit-shift during training is intentional. The integer >> operator is not differentiable, so gradients cannot flow through it. Using floating-point division (which has the same mathematical result for positive powers of 2) allows the backpropagation algorithm to update the weights correctly during training.*

4. Training Setup & Dependencies

Hardware Requirements

- A computer with Python 3.8 or later installed.
- A CUDA-compatible GPU is recommended for faster training but is not required — the code automatically falls back to CPU.
- At least 4 GB of RAM. 8 GB recommended.

Software Dependencies

Package	Version	Purpose
`torch (PyTorch)`	2.0+	Neural network training framework
`torchvision`	0.15+	MNIST dataset download and transforms
`numpy`	1.x+	Array operations and .npz export

Installation

Install all dependencies with pip:

```
pip install torch torchvision numpy
```

If you have a CUDA GPU, install the GPU-enabled version of PyTorch by following the instructions at pytorch.org/get-started. The training code will automatically detect and use the GPU.

Required Files

File	Description
`lenet_qat_train.py`	The training script. Run this first.
`lenet_numpy_inference.py`	The software inference/verification script. Run this after training.
`./data/ (auto-downloaded)`	MNIST dataset. Downloaded automatically on first run.
`lenet_int8_fpga_working.npz (output)`	Exported INT8 weights file. Produced by the training script. This is what gets deployed to the FPGA.

Running the Training Script

To train the model, simply run:

```
python lenet_qat_train.py
```

Training runs for 20 epochs. On a modern CPU this takes approximately 10 to 30 minutes. On a GPU it typically completes in under 5 minutes. You will see per-epoch progress printed to the console:

```
Epoch [1/20]  Loss: 48.312  Train Acc: 91.23%
Epoch [2/20]  Loss: 31.745  Train Acc: 94.87%
...
Epoch [20/20] Loss: 12.003  Train Acc: 98.21%
Test Accuracy: 97.95%
Exported: lenet_int8_fpga_working.npz
```

> **NOTE:** *The MNIST dataset (approximately 60,000 training images, 10,000 test images) will be downloaded automatically the first time you run the script. This requires an internet connection and takes a few minutes. Subsequent runs use the cached data in ./data/.*

5. Code Walkthrough — Training

This section walks through every component of the training script in the order a beginner would need to understand them. Each subsection explains what the code does and, crucially, why it must be written that way for this hardware.

5.1 INT8 Straight-Through Estimator (STE)

The first major component is the Int8STE class. This is the heart of quantisation-aware training.

```
class Int8STE(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x):
        return torch.round(x).clamp(QMIN, QMAX)

    @staticmethod
    def backward(ctx, grad_output):
        return grad_output

def qint8(x):
    return Int8STE.apply(x)
```

What Int8STE Does

This class defines a custom mathematical operation that has different behaviour during the forward pass (when making predictions) and the backward pass (when computing gradients for learning).

Pass	What happens
`Forward`	The input tensor x is rounded to the nearest integer and then clipped to the range [-128, 127]. This simulates INT8 quantisation — exactly what the FPGA hardware does.
`Backward`	The gradient is passed through unchanged — as if the rounding operation never happened. This is the Straight-Through trick.

Why This Is Necessary: The Rounding Problem

Rounding (torch.round) is a step function — it is flat almost everywhere and has undefined or zero gradients at all points. If PyTorch tried to compute the true gradient through rounding, it would always get zero, and the model weights would never update.

> **KEY CONCEPT:** *The Straight-Through Estimator is a trick: pretend during backpropagation that the rounding never happened, and pass the gradient through as if it were an identity function. This allows the model to learn despite the non-differentiable quantisation.*

How qint8() Is Used

The qint8() function is called in three places throughout the model:

1. On the input image at the very start of the forward pass.
2. On the layer weights inside each QuantConv2d and QuantLinear layer.
3. On the input activations to each layer (the output of the previous layer).

This means that at every point in the computation, values are constrained to the INT8 range — exactly as they will be on the FPGA hardware.

> **IMPORTANT:** *Calling qint8() on both inputs and weights before every layer is not optional. Removing any of these calls will produce a model whose activations can grow beyond INT8 range during training, leading to a mismatch with the hardware and incorrect predictions on the FPGA.*

5.2 FPGA Requantisation Function

After each layer computes its INT32 accumulation (the sum of INT8 multiplications), the result must be rescaled back to INT8. This is done by the fpga_requant() function.

```
def fpga_requant(acc):
    acc = acc * SCALE_FACTOR          # Multiply by 258
    acc = acc + ROUND_FACTOR          # Add 16384 (rounding offset)
    acc = acc / (2 ** SHIFT)          # Divide by 32768 (floating-point)
    return acc                        # Clipping done by qint8() later
```

Relationship to the Hardware

This function is the software mirror of a fixed hardware operation inside the FPGA accelerator. The hardware performs the exact same three steps, but uses integer arithmetic:

Training (fpga_requant)	FPGA Hardware
`acc * SCALE_FACTOR`	acc * 258 (integer multiply)
`acc + ROUND_FACTOR`	acc + 16384 (integer add)
`acc / (2 ** SHIFT)`	acc >> 15 (integer right-shift)
`Clip handled by next qint8()`	Clip to [-128, 127] in hardware

Why Division Instead of Bit-Shift in Training

The integer right-shift operator (>>) used in the FPGA cannot be differentiated — PyTorch cannot compute gradients through it. However, for positive powers of 2, dividing by 2^N gives the same numerical result as shifting right by N bits. Since 2^15 = 32768, the division and the shift produce identical values.

Using division instead of shift during training therefore:

- Produces the same numerical output as the hardware.
- Allows gradients to flow through during backpropagation (the division is differentiable).

> **NOTE:** *The clipping step (clip to [-128, 127]) is not performed inside fpga_requant(). Instead, it is applied by the next call to qint8() at the beginning of the following layer. The result is the same — each layer's output is clipped to INT8 before being used as the next layer's input.*

5.3 QuantConv2d and QuantLinear Layers

These two classes replace the standard PyTorch nn.Conv2d and nn.Linear layers. They add quantisation at three specific points in the computation.

QuantConv2d

```
class QuantConv2d(nn.Module):
    def __init__(self, in_ch, out_ch, k):
        super().__init__()
        self.weight = nn.Parameter(torch.randn(out_ch, in_ch, k, k) * 0.5)
        self.bias   = nn.Parameter(torch.zeros(out_ch))

    def forward(self, x):
        x_q  = qint8(x)              # 1. Quantise input to INT8
        w_q  = qint8(self.weight)    # 2. Quantise weights to INT8
        acc  = F.conv2d(x_q, w_q, bias=None)  # 3. Convolution (no bias yet)
        acc  = acc + self.bias.view(1, -1, 1, 1)  # 4. Add bias (float)
        return fpga_requant(acc)     # 5. Rescale back to INT8 range
```

The five steps match what the FPGA hardware does:

Step	What happens and why
`1. qint8(x)`	Input activations are quantised. This ensures the input is always in INT8 range, matching the FPGA which stores activations in BRAMs as INT8.
`2. qint8(self.weight)`	Kernel weights are quantised. The FPGA stores weights as INT8, so the training must also use INT8 weights for the multiply.
`3. F.conv2d (no bias)`	INT8 x INT8 multiply-accumulate. The bias is intentionally excluded here because the FPGA adds bias separately after accumulation.
`4. Add bias (float)`	Bias is added as floating point during training (stored as float32/int32 in hardware). Bias values are NOT quantised to INT8 — they remain as int32.
`5. fpga_requant(acc)`	The INT32 accumulation is rescaled back to INT8 range using the fixed hardware constants.

QuantLinear

```
class QuantLinear(nn.Module):
    def __init__(self, in_f, out_f):
        super().__init__()
        self.weight = nn.Parameter(torch.randn(out_f, in_f) * 0.5)
        self.bias   = nn.Parameter(torch.zeros(out_f))

    def forward(self, x):
        x_q = qint8(x)               # Quantise input
        w_q = qint8(self.weight)     # Quantise weights
        acc = F.linear(x_q, w_q, bias=None)  # Matrix multiply (no bias)
        acc = acc + self.bias        # Add bias separately
        return fpga_requant(acc)     # Rescale to INT8 range
```

QuantLinear is identical in structure to QuantConv2d, but uses a matrix multiplication (F.linear) instead of a convolution (F.conv2d). It is used for the three fully-connected layers (FC1, FC2, FC3).

Weight Initialisation

Both layers initialise their weights with:

```
torch.randn(shape) * 0.5
```

The scaling factor of 0.5 is important. Standard PyTorch initialisers (like Kaiming or Xavier) are designed for floating-point networks and may produce initial weights outside the useful INT8 range. Scaling by 0.5 keeps initial weights small and well within [-128, 127] after the first qint8() call, which helps training converge faster.

5.4 LeNet_INT8_QAT Model Architecture

The full model is defined in the LeNet_INT8_QAT class:

```
class LeNet_INT8_QAT(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = QuantConv2d(1, 6, 3)       # 1 input ch, 6 output ch, 3x3 kernel
        self.conv2 = QuantConv2d(6, 16, 3)      # 6 input ch, 16 output ch, 3x3 kernel
        self.fc1   = QuantLinear(16 * 5 * 5, 120)
        self.fc2   = QuantLinear(120, 84)
        self.fc3   = QuantLinear(84, 10)

    def forward(self, x):
        x = qint8(x)                    # CRITICAL: quantise the raw input
        x = self.conv1(x)
        x = F.relu(x)
        x = F.max_pool2d(x, 2)
        x = self.conv2(x)
        x = F.relu(x)
        x = F.max_pool2d(x, 2)
        x = x.view(x.size(0), -1)       # Flatten to 1D vector
        x = self.fc1(x)
        x = F.relu(x)
        x = self.fc2(x)
        x = F.relu(x)
        x = self.fc3(x)
        return x
```

Layer-by-Layer Dimensions

Layer	Output Shape	Notes
`Input (28x28 image)`	`(B, 1, 28, 28)`	Scaled to [0, 127], then qint8() clips to INT8
`Conv1 (3x3, 6 filters)`	`(B, 6, 26, 26)`	No padding: 28-3+1=26. ReLU applied after.
`MaxPool2d (2x2)`	`(B, 6, 13, 13)`	Halves each spatial dimension
`Conv2 (3x3, 16 filters)`	`(B, 16, 11, 11)`	No padding: 13-3+1=11. ReLU applied after.
`MaxPool2d (2x2)`	`(B, 16, 5, 5)`	Halves each spatial dimension
`Flatten`	`(B, 400)`	16 * 5 * 5 = 400 values
`FC1 (400 -> 120)`	`(B, 120)`	ReLU applied after
`FC2 (120 -> 84)`	`(B, 84)`	ReLU applied after
`FC3 (84 -> 10)`	`(B, 10)`	10 output scores, one per digit class

The Critical Input Quantisation Line

The very first line of forward() is:

```
x = qint8(x)   # CRITICAL: quantise the raw input
```

> **IMPORTANT:** *This line is not optional. The FPGA firmware quantises the input image to INT8 before sending it to hardware (using the same clamp to [-128, 127]). If you remove this line from training, the model will see floating-point inputs during training but INT8 inputs during inference, causing a systematic mismatch and poor accuracy on the FPGA.*

5.5 Dataset Preparation

The MNIST dataset is loaded with a custom transform that scales pixel values to match exactly what the FPGA firmware sends to the hardware:

```
transform = transforms.Compose([
    transforms.ToTensor(),                          # Converts PIL image to
                                                    # float32 tensor in [0.0, 1.0]
    transforms.Lambda(lambda x: x * 127.0)         # Scale to [0.0, 127.0]
])
```

Why x * 127 Instead of Standard Normalisation

Standard MNIST training typically uses:

```
# Standard approach (NOT used here):
transforms.Normalize(mean=(0.1307,), std=(0.3081,))
```

This produces zero-mean, unit-variance inputs centred around 0. For floating-point networks this is beneficial. However, the FPGA firmware does NOT apply this normalisation — it scales the image to [0, 127] and quantises to INT8. Training with standard normalisation would cause every test image to look different to the model compared to what the FPGA sends, leading to incorrect predictions.

> **KEY CONCEPT:** *The training transform must exactly replicate what the FPGA firmware does to the input image. Since the firmware multiplies pixel values by 127 and clips to INT8, the training transform must do the same. This is one of the most important hardware-software consistency requirements.*

DataLoader Configuration

```
train_loader = DataLoader(train_set, batch_size=64,  shuffle=True)
test_loader  = DataLoader(test_set,  batch_size=256, shuffle=False)
```

Parameter	Explanation
`batch_size=64 (train)`	64 images processed simultaneously during training. Larger batches = faster training but more memory. 64 is a safe default.
`batch_size=256 (test)`	Larger batch for evaluation since no gradients are computed, so memory usage is lower.
`shuffle=True (train)`	Randomly shuffles training data each epoch to prevent the model from learning the order of samples.

`shuffle=False (test)`	Test data is evaluated in fixed order for reproducible accuracy measurements.

5.6 Training Loop

```
model     = LeNet_INT8_QAT().to(DEVICE)
optimizer = torch.optim.Adam(model.parameters(), lr=0.002)
criterion = nn.CrossEntropyLoss()
EPOCHS    = 20

for epoch in range(EPOCHS):
    model.train()
    for imgs, labels in train_loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        out  = model(imgs)
        loss = criterion(out, labels)
        loss.backward()
        optimizer.step()
```

Training Configuration Explained

Setting	Explanation
`Adam optimiser, lr=0.002`	Adam is a good default optimiser. The learning rate 0.002 is slightly higher than typical (0.001) to compensate for the additional noise introduced by quantisation during training.
`CrossEntropyLoss`	Standard loss function for classification. Measures how far the model's 10 output scores are from the correct one-hot label.
`EPOCHS = 20`	The model trains for 20 full passes over all 60,000 training images. This is sufficient for MNIST with QAT. More epochs rarely improve accuracy significantly.
`model.to(DEVICE)`	Moves the model to GPU if available, otherwise stays on CPU. DEVICE is set automatically at the top of the script.
`optimizer.zero_grad()`	Clears accumulated gradients from the previous batch. Must be called before every backward() call.

What Happens to Gradients Through Quantisation

During the backward pass, the STE allows gradients to flow through all qint8() calls as if they were identity functions. The fpga_requant() division is fully differentiable. This means the gradient flow looks like:

```
FC3 output --> FC3 weights
           --> FC2 output (through fpga_requant division, fully differentiable)
           --> FC2 weights
           --> ... (through qint8 STE, straight-through)
           --> Conv1 weights
```

> **NOTE:** *Despite all the quantisation, the training loss and accuracy curves typically look similar to standard floating-point training. The STE approximation is surprisingly effective for MNIST-scale tasks.*

5.7 Exporting Weights to .npz

After training, the export_int8() function converts the learned floating-point weights to hard INT8 values and saves everything to a .npz file:

```
def export_int8(model):
    export = {}

    def q8(x):    # Quantise to INT8 numpy array
        return torch.round(x).clamp(QMIN, QMAX)
                  .detach().cpu().numpy().astype(np.int8)

    def q32(x):   # Round to INT32 numpy array (for biases)
        return torch.round(x).detach().cpu().numpy().astype(np.int32)

    export['conv1_w'] = q8(model.conv1.weight)   # INT8 kernels
    export['conv1_b'] = q32(model.conv1.bias)    # INT32 biases
    # ... (same for conv2, fc1, fc2, fc3)

    export['ScaleFactor'] = np.array(SCALE_FACTOR, dtype=np.int32)  # 258
    export['RoundFactor'] = np.array(ROUND_FACTOR, dtype=np.int32)  # 16384
    export['Shift']       = np.array(SHIFT,        dtype=np.int32)  # 15

    return export

np.savez('lenet_int8_fpga_working.npz', **export_int8(model))
```

Why Weights Are INT8 but Biases Are INT32

This matches the FPGA hardware design:

- Convolution kernels and FC weight matrices are stored as INT8 in the FPGA BRAMs. Using INT8 saves memory and allows the hardware to process 16 values in parallel in a single 128-bit BRAM word.
- Biases are stored as INT32. Because bias values can be larger in magnitude (they are added after accumulation of many INT8 products) and there are far fewer of them (one per output channel), the extra memory cost of INT32 is acceptable and avoids precision loss.

Why the Constants Are Also Saved

The three constants SCALE_FACTOR, ROUND_FACTOR, and SHIFT are saved inside the .npz file alongside the weights. This is important because:

- The FPGA firmware reads these constants from the .npz file when loading the model.
- It ensures that the firmware always uses the same rescaling parameters that were used during training, even if someone changes the constants and retrains.
- It makes the .npz file self-contained — everything needed to run inference is in one file.

> **NOTE:** *After the export, verify the file was created by checking that lenet_int8_fpga_working.npz exists in the current directory. The file is typically around 100–200 KB.*

6. Code Walkthrough — Software Inference

The software inference script (lenet_numpy_inference.py) is a verification tool. It runs the exported INT8 weights through a pure NumPy implementation of the LeNet pipeline that mirrors the FPGA hardware exactly. If this script achieves good accuracy, the exported weights are correct and safe to deploy to the FPGA.

> **KEY CONCEPT:** *Think of this script as a software simulation of the FPGA. It uses integer arithmetic (INT8 multiply, INT32 accumulate, bit-shift requantise) in the same sequence the hardware uses, so any accuracy measured here should match what the FPGA achieves.*

6.1 Loading Weights

```
data = np.load('lenet_int8_fpga_working.npz')

conv1_w = data['conv1_w']   # shape: (6, 1, 3, 3),  dtype: int8
conv1_b = data['conv1_b']   # shape: (6,),           dtype: int32
# ... (same for conv2, fc1, fc2, fc3)

SCALE_FACTOR = int(data['ScaleFactor'])  # 258
ROUND_FACTOR = int(data['RoundFactor'])  # 16384
SHIFT        = int(data['Shift'])        # 15
```

All weights are loaded directly from the .npz file. The quantisation constants are also read from the same file, ensuring they always match the training configuration.

6.2 INT8 Convolution and Helper Functions

The script defines four helper functions that implement each operation in hardware-faithful integer arithmetic.

conv2d_int8()

```
def conv2d_int8(x, w, b):
    N, C, H, W = x.shape
    K, _, KH, KW = w.shape
    OH = H - KH + 1
    OW = W - KW + 1
    out = np.zeros((N, K, OH, OW), dtype=np.int32)  # Accumulate in INT32
    for n in range(N):
        for k in range(K):
            for c in range(C):
                for i in range(OH):
                    for j in range(OW):
                        out[n,k,i,j] += np.sum(
                            x[n,c,i:i+KH,j:j+KW].astype(np.int32) *
                            w[k,c].astype(np.int32)
                        )
            out[n,k] += b[k]   # Add INT32 bias
    return out
```

Note the .astype(np.int32) casts before multiplication. This is critical: multiplying two int8 values without upcasting would overflow since INT8 can only hold -128 to 127, but INT8 * INT8 can produce values up to 127 * 127 = 16129.

> **IMPORTANT:** *This pure Python convolution loop is very slow — O(N x K x C x OH x OW x KH x KW) operations. It is intentionally written this way for clarity and hardware-faithfulness, not speed. For the full 10,000-image test set this may take 10–30 minutes. Use MAX_SAMPLES = 1000 for a quick verification run.*

hw_quant()

```
def hw_quant(acc):
    acc = acc * SCALE_FACTOR    # Multiply by 258
    acc = acc + ROUND_FACTOR    # Add 16384
    acc = acc >> SHIFT          # Integer right-shift by 15
    acc = np.clip(acc, QMIN, QMAX)
    return acc.astype(np.int8)
```

This is the integer version of fpga_requant() from training. The key difference is that this uses the >> bitwise shift operator instead of floating-point division, exactly as the FPGA hardware does. The output is cast to int8.

relu() and maxpool2d()

```
def relu(x):
    return np.maximum(x, 0)    # Zero out negative values

def maxpool2d(x, k=2):
    # Takes the maximum value in each 2x2 window
    # Reduces spatial dimensions by half
    out = np.zeros((N, C, H//k, W//k), dtype=np.int8)
    for each 2x2 window:
        out = np.max(window)
    return out
```

Both operations work directly on INT8 values. ReLU simply clips negatives to zero. MaxPool2d selects the largest value in each 2x2 block — both operations are naturally INT8-compatible since they do not change the scale of the data.

6.3 Full Inference Pipeline

The inference pipeline processes images from the MNIST test set in batches:

```
# Input: imgs already scaled to [0,127] by transform, cast to int8
imgs = imgs.numpy().astype(np.int8)

# Conv1: INT8 in, INT32 accumulate, requantise to INT8, ReLU, MaxPool
x = conv2d_int8(imgs, conv1_w, conv1_b)  # INT32 output
x = hw_quant(x)                           # Back to INT8
x = relu(x)                               # Still INT8
x = maxpool2d(x)                          # Still INT8

# Conv2: same sequence
x = conv2d_int8(x, conv2_w, conv2_b)
x = hw_quant(x)
x = relu(x)
x = maxpool2d(x)

# Flatten: 16 channels x 5x5 = 400 values
x = x.reshape(x.shape[0], -1)

# FC1, FC2, FC3: same sequence (no maxpool)
x = linear_int8(x, fc1_w, fc1_b)
x = hw_quant(x)
x = relu(x)
# ... FC2 ...
x = linear_int8(x, fc3_w, fc3_b)
x = hw_quant(x)

# Prediction: digit with highest score
preds = np.argmax(x, axis=1)
```

Comparison with the FPGA Firmware Pipeline

The software inference pipeline and the FPGA firmware pipeline perform the same operations in the same order. The table below shows the correspondence:

Software Inference (NumPy)	FPGA Firmware (Python Driver)
`conv2d_int8(imgs, conv1_w, conv1_b)`	write_fmap_to_cdma + write_kernel_to_cdma + run()
`hw_quant(x) using >> SHIFT`	Hardware ControlSig: SCALE_FACTOR * SHIFT in logic
`relu(x)`	driver.configure(relu_en=1)
`maxpool2d(x)`	driver.configure(maxpool_en=1) + run()
`linear_int8(x, fc1_w, fc1_b)`	Runs in Python software (same in both)
`np.argmax(x)`	np.argmax(x) in firmware predict_digit()

6.4 Per-Digit Accuracy Reporting

The inference script tracks and prints accuracy separately for each digit class (0–9). This is useful for diagnosing which digits the model struggles with:

```
digit_correct = np.zeros(10, dtype=int)  # How many of each digit were correct
digit_total   = np.zeros(10, dtype=int)  # How many of each digit were seen

# After each batch, update per-digit counts:
for d in range(10):
    idx = labels == d               # Boolean mask for digit d
    digit_total[d]   += np.sum(idx)
    digit_correct[d] += np.sum(preds[idx] == d)
```

A running accuracy summary is printed after each batch, so you can see progress without waiting for all batches to complete:

```
[Batch 00] Samples:  256 | Running Acc:  97.27%
 Digit 0: 100.00% (28/28)
 Digit 1: 100.00% (32/32)
 Digit 2:  96.67% (29/30)
 ...
--------------------------------------------------
```

The MAX_SAMPLES Setting

```
MAX_SAMPLES = 1000   # Set to None for the full 10,000-image test set
```

Because the pure NumPy convolution is slow, the script defaults to testing on the first 1,000 images only. This takes a few minutes and gives a reliable accuracy estimate. Set MAX_SAMPLES = None to run the full 10,000-image evaluation.

> **NOTE:** *An accuracy above 95% on 1,000 samples means the exported weights are correct and the model is ready for FPGA deployment. The FPGA firmware should achieve similar accuracy when running the same test images.*

7. API Reference

This section provides a concise reference for every class and function in both scripts.

7.1 Training Script (lenet_qat_train.py)

Name	Type	Description
`Int8STE`	torch.autograd.Function	Custom autograd function implementing INT8 quantisation in forward and identity (STE) in backward.
`qint8(x)`	Function	Applies Int8STE. Rounds and clips x to [-128, 127]. Call on inputs and weights before each layer.
`fpga_requant(acc)`	Function	Rescales INT32 accumulation to INT8 range using SCALE_FACTOR, ROUND_FACTOR, SHIFT. Uses floating-point division for gradient flow.
`QuantConv2d(in_ch, out_ch, k)`	nn.Module	Hardware-faithful quantised 2D convolution. Quantises inputs and weights to INT8, accumulates in float32, applies fpga_requant.
`QuantLinear(in_f, out_f)`	nn.Module	Hardware-faithful quantised fully-connected layer. Same quantisation scheme as QuantConv2d.
`LeNet_INT8_QAT()`	nn.Module	Full LeNet-5 model using QuantConv2d and QuantLinear layers. Includes input quantisation in forward().
`export_int8(model)`	Function	Extracts weights from trained model as numpy arrays. Weights cast to int8, biases to int32. Returns dict for np.savez.

7.2 Software Inference Script (lenet_numpy_inference.py)

Function Signature	Returns	Description
`conv2d_int8(x, w, b)`	`ndarray int32`	Performs 2D valid convolution of x with weights w and bias b. Accumulates in INT32. Input/weights cast to int32 before multiply.
`linear_int8(x, w, b)`	`ndarray int32`	Performs matrix multiply x @ w.T + b. All operands cast to int32.
`hw_quant(acc)`	`ndarray int8`	Applies the hardware rescaling formula: multiply by SCALE_FACTOR, add ROUND_FACTOR, right-shift by SHIFT, clip to [-128, 127].

`relu(x)`	`ndarray int8`	Returns np.maximum(x, 0). Works on any integer dtype.
`maxpool2d(x, k=2)`	`ndarray int8`	2D max-pooling with window size k x k and stride k. No padding. Halves each spatial dimension.

7.3 Global Constants (Both Scripts)

Constant	Value	Purpose
`SCALE_FACTOR`	`258`	Fixed multiplier in the INT32-to-INT8 rescaling pipeline. Must match FPGA hardware.
`ROUND_FACTOR`	`16384`	Rounding offset (= 2^14) added before the right-shift. Must match FPGA hardware.
`SHIFT`	`15`	Number of right-shift bits (divide by 2^15 = 32768). Must match FPGA hardware.
`QMIN`	`-128`	Minimum value for signed 8-bit integer. Used in clipping.
`QMAX`	`127`	Maximum value for signed 8-bit integer. Used in clipping.
`DEVICE`	`cuda or cpu`	Set automatically. Moves model and data to GPU if available.

8. Running the Full Flow

This section gives you the complete end-to-end sequence from training a model to running it on the FPGA, with all steps in the correct order.

Step 1 — Train the Model

On your PC or laptop (not the FPGA board):

```
python lenet_qat_train.py
```

Wait for 20 epochs to complete. You should see Test Accuracy above 95%. The file lenet_int8_fpga_working.npz is created in the current directory.

Step 2 — Verify with Software Inference

Before touching the FPGA, verify the exported weights are correct:

```
python lenet_numpy_inference.py
```

This runs the first 1,000 MNIST test images through the NumPy integer pipeline. You should see running accuracy above 95%. If accuracy is much lower, do not proceed to FPGA deployment — retrain the model.

Step 3 — Copy Weights to the PYNQ Board

Transfer the .npz file to the PYNQ board. You can do this via the Jupyter file upload interface or using scp from a terminal:

```
# From your PC terminal:
scp lenet_int8_fpga_working.npz xilinx@<board-ip>:~/jupyter_notebooks/LeNet_data/
```

> **NOTE:** *The firmware manual specifies that the file must be at LeNet_data/lenet_int8_fpga_working_95.npz relative to the notebook. Rename the file if needed, or update the path in the notebook.*

Step 4 — Run Inference on the FPGA

Open the Jupyter Notebook on the PYNQ board and run the firmware as described in the companion firmware manual. The model.predict_digit() call will load the weights from the .npz file and run inference using the FPGA hardware.

Summary Flow Diagram

```
                    [ lenet_qat_train.py ]
              Train 20 epochs on MNIST with INT8 QAT
                                |
                                v
             [ lenet_int8_fpga_working.npz ]
        INT8 weights + INT32 biases + quantisation constants
                        |                     |
                        v                     v
[ lenet_numpy_inference.py ]     [ PYNQ-ZU FPGA Firmware ]
  Verify accuracy on PC          Run hardware inference on board
   (must be >95% before deploy)   (predict_digit() in notebook)
```

9. Troubleshooting

Problem	Likely Cause and Solution
Training accuracy stays below 90% after 20 epochs	Check that the input transform uses x * 127.0 and NOT standard MNIST normalisation. Also ensure qint8() is called at the very start of forward(). Both are common sources of poor QAT convergence.
Test accuracy is high in training but drops on the FPGA	The most common cause is a mismatch in input preprocessing. The FPGA firmware must apply the same [0, 127] scaling as the training transform. Also verify that the constants in the .npz match the hardware values (258, 16384, 15).
Software inference accuracy is much lower than training test accuracy	Check that hw_quant() uses integer right-shift (>>) not floating-point division. The STE training uses division for gradient flow, but the software inference must use the integer shift to match the hardware exactly.
ModuleNotFoundError: No module named torch	PyTorch is not installed. Run: pip install torch torchvision
Software inference is extremely slow	The pure Python convolution loops are intentionally slow. Use MAX_SAMPLES = 1000 (the default) for verification. Set to None only if you need the exact 10,000-image accuracy figure.
lenet_int8_fpga_working.npz not found when running inference script	The training script must be run first to generate the .npz file. Ensure both scripts are in the same directory, or update the file path at the top of the inference script.
FPGA gives different predictions than the software inference	Verify the BRAM addresses in the firmware match the bitstream. Also check that the .npz file transferred to the board is the same file verified by the software inference script (use file size or a checksum to confirm).
Accuracy drops when changing kernel size from 3x3 to 5x5	The model architecture in LeNet_INT8_QAT currently uses 3x3 kernels. If you change to 5x5, the FC1 input size changes from 16*5*5=400 to 16*4*4=256 (for 28x28 input). Update the fc1 definition and retrain. The FPGA bitstream also needs to support the new kernel size.

The Most Common Mistake

> **IMPORTANT:** *The single most common mistake when retraining is using standard MNIST normalisation (mean=0.1307, std=0.3081) instead of the x * 127.0 scaling. Everything else can look fine — training accuracy, test accuracy, even the .npz file — but the FPGA will produce wrong predictions because every input image looks different to what the model was trained on. Always double-check the transform.*